

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО

ОТЧЁТ
ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»
по теме:
Д34: ТЕХНИЧЕСКИЙ ДИЗАЙН МИКРОСЕРВИСНОЙ АРХИТЕКТУРЫ
(СЕРВИС АРЕНДЫ НЕДВИЖИМОСТИ)

Проверил:

подпись, дата

Добряков Д. И.

Выполнил:

Группа

подпись, дата

Субботин Александр

1 Задача

Тема: Сервис аренды недвижимости.

Требуется:

- разделить монолитное бэкенд-приложение на микросервисы (database-per-service);
- спроектировать архитектуру: взаимосвязи и способы взаимодействия;
- спроектировать разделение БД на обособленные БД;
- описать эндпоинты межсервисного взаимодействия (OpenAPI);
- описать варианты запросов, ответов и ошибок.

Стек: Go, Gin, PostgreSQL, REST, JWT, Docker Compose.

2 Ход работы

2.1 1. Анализ монолита и выделение bounded-контекстов

Монолитное приложение (JP1) содержит следующие доменные области:

- Аутентификация и пользователи
- Объекты недвижимости, фотографии, удобства, избранное
- Бронирования, отзывы, оплаты
- Чаты и сообщения

Каждая область имеет слабую связанность с остальными и может быть выделена в отдельный сервис.

2.2 2. Микросервисная архитектура

Монолит разделяется на 4 микросервиса и API Gateway:

Сервис	Порт	Ответственность
user-service	8081	Регистрация, вход, JWT, профиль пользователя
property-service	8082	Объекты недвижимости, фото, удобства, избранное
booking-service	8083	Бронирования, отзывы, оплаты
chat-service	8084	Чаты, сообщения между пользователями
api-gateway	8080	Маршрутизация запросов к сервисам

2.2.1 2.1 Принцип database-per-service

Все сервисы подключаются к единому экземпляру PostgreSQL, но каждый работает с собственной базой данных:

Сервис	БД	Таблицы
user-service	user_db	users
property-service	property_db	properties, property_photos, amenities, property_amenities, favorites
booking-service	booking_db	bookings, reviews, payments

chat-service	chat_db	chats, chat_participants, messages
--------------	---------	------------------------------------

Один PostgreSQL-контейнер содержит 4 логически изолированных базы. При инициализации скрипт `01_create_databases.sh` создаёт все базы и применяет миграции.

Внешние ключи между таблицами разных сервисов заменены на UUID-ссылки без FK-constraint. Целостность обеспечивается на уровне приложения через межсервисные вызовы.

2.2.2 2.2 Способы взаимодействия

Синхронное взаимодействие (HTTP REST):

- booking-service → property-service: получение данных об объекте (цена, владелец) для создания бронирования и проверки прав.

API Gateway:

- Принимает все входящие запросы на порту 8080.
- Проксирует запросы к нужному сервису на основе URL-пути.
- Передаёт заголовок Authorization без изменений.

Аутентификация:

- Каждый сервис самостоятельно валидирует JWT-токен (общий секрет JWT_SECRET).
- user-service выдаёт токены при логине.

2.3 3. Разделение базы данных

2.3.1 3.1 user_db

users

- id UUID, PK
- email text, unique, not null
- phone text, unique, null
- password_hash text, not null
- full_name text, not null
- is_active bool, default true

- created_at timestamptz
- updated_at timestamptz

2.3.2 3.2 property_db

properties

- id UUID, PK
- owner_id UUID, not null (*ссылка на user_db.users.id, без FK*)
- title, description, property_type, price_per_night, currency, city, address, lat, lon, rooms, beds, max_guests, area_m2, check_in_time, check_out_time, rules, status
- created_at, updated_at timestamptz

property_photos — фотографии объекта (FK → properties.id)

amenities — справочник удобств

property_amenities — связь many-to-many

favorites

- user_id UUID (*ссылка на user_db, без FK*)
- property_id UUID, FK → properties.id
- PK(user_id, property_id)

2.3.3 3.3 booking_db

bookings

- id UUID, PK
- property_id UUID (*ссылка на property_db, без FK*)
- tenant_id UUID (*ссылка на user_db, без FK*)
- start_date, end_date, guests_count, price_total, status
- created_at, updated_at

reviews

- booking_id UUID, unique, FK → bookings.id
- property_id UUID (*ссылка на property_db*)
- author_id UUID (*ссылка на user_db*)

– rating, text, created_at

payments — факт оплаты (FK → bookings.id)

2.3.4 3.4 chat_db

chats — чат (опциональные property_id, booking_id как UUID без FK)

chat_participants — участники чата (user_id UUID без FK)

messages — сообщения (sender_id UUID без FK)

2.4 4. Маршрутизация API Gateway

Метод	Путь	Целевой сервис
POST	/auth/register	user-service
POST	/auth/login	user-service
GET	/users/me	user-service
GET/ POST/ PATCH/ DELETE	/properties/*	property-service
GET	/users/ me/properties	property-service
GET/ POST/ DELETE	/favorites/*	property-service
POST/ GET/ PATCH	/bookings/*	booking-service
GET	/users/me/owner/ bookings	booking-service
POST	/reviews	booking-service
GET/POST	/chats/*	chat-service

2.5 5. Межсервисное взаимодействие (Internal API)

2.5.1 5.1 property-service → internal API

GET /internal/properties/{propertyId}

Используется booking-service для получения информации об объекте.

Ответ 200:

```
{
  "id": "uuid",
  "owner_id": "uuid",
  "price_per_night": 5000.00,
  "title": "...",
  "status": "active"
}
```

Ответ 404:

```
{"code": 404, "message": "property not found"}
```

—

GET /internal/properties?owner_id={userId}

Возвращает список ID объектов владельца. Используется booking-service для GET /users/me/owner/bookings.

Ответ 200:

```
{"property_ids": ["uuid1", "uuid2"]}
```

Ответ 400:

```
{"code": 400, "message": "owner_id is required"}
```

2.6 6. Публичные эндпоинты по сервисам

2.6.1 6.1 user-service

Метод	Путь	Описание / ошибки
POST	/auth/register	201 User; 400; 409 email занят
POST	/auth/login	200 {access_token}; 400; 401
GET	/users/me	200 User; 401; 404

2.6.2 6.2 property-service

Метод	Путь	Описание / ошибки
GET	/properties	200 {items, pagination}; 400

POST	/properties	201 Property; 400; 401
GET	/properties/{id}	200 Property; 404
PATCH	/properties/{id}	200 Property; 401; 403; 404
DELETE	/properties/{id}	204; 401; 403; 404
GET	/users/me/properties	200 {items, pagination}; 401
GET	/favorites	200 Property[]; 401
POST	/favorites/{id}	201 Favorite; 401; 404; 409
DELETE	/favorites/{id}	204; 401

2.6.3 6.3 booking-service

Метод	Путь	Описание / ошибки
POST	/bookings	201 Booking; 400; 401; 404; 409; 422
GET	/bookings	200 {items, pagination}; 401
GET	/bookings/{id}	200 Booking; 401; 403; 404
PATCH	/bookings/{id}	200 Booking; 400; 401; 403; 404
GET	/users/me/owner/bookings	200 {items, pagination}; 401
POST	/reviews	201 Review; 400; 401; 403; 404; 409

2.6.4 6.4 chat-service

Метод	Путь	Описание / ошибки
GET	/chats	200 {items, pagination}; 401
GET	/chats/{id}/messages	200 {items, pagination}; 401; 403
POST	/chats/{id}/messages	201 Message; 400; 401; 403

2.7 7. Формат ошибок (единый)

```
{
  "code": 404,
  "message": "property not found",
  "details": null
}
```

2.8 8. Шаги для миграции монолита

1. Создать 4 отдельных Go-модуля (по сервису) с собственным go.mod.
2. Разнести таблицы по 4 БД, убрать кросс-сервисные FK, заменить на UUID-ссылки.

3. Вынести handler/repository/model в каждый сервис, оставив только нужные.
4. Реализовать HTTP-клиент в booking-service для обращения к property-service (internal API).
5. Реализовать API Gateway (reverse proxy) для единой точки входа.
6. Написать docker-compose.yaml с единым PostgreSQL (4 БД), 4 сервисами и gateway.
7. Каждый сервис валидирует JWT самостоятельно (общий секрет).

3 Вывод

Спроектирована микросервисная архитектура сервиса аренды недвижимости. Монолит разделён на 4 сервиса (user, property, booking, chat) с отдельными логическими базами данных в рамках единого PostgreSQL (database-per-service). Описаны публичные и внутренние эндпоинты, способы межсервисного взаимодействия (синхронный HTTP REST), маршрутизация через API Gateway и конкретные шаги миграции.